BRETAS
IMÓVEIS
BRASIL E EXTERIOR

Imóveis Bretas

Protótipo (Esqueleto) – Desenvolvimento de Aplicação Web com Spring Boot

Vitrine de imóveis com painel administrativo

Integrantes: Felipe · Jeff · Ruan · Anna · Jorge

Tema e Problema de Negócio

Divulgação

A imobiliária precisa divulgar imóveis para compra e aluguel na internet.

Busca

Clientes precisam filtrar por tipo, localização, preço e cômodos.

Gestão

Corretores precisam cadastrar, editar e remover anúncios com segurança.

Solução: site público + painel administrativo + API REST com banco de dados relacional.

Regras de Negócio

- Imóvel tem tipo (Casa, Apartamento, Terreno) e situação (Compra ou Aluguel)
- Valor e área são obrigatórios e maiores que zero
- E-mail de usuário é único; senha salva criptografada (BCrypt)
- Cadastro público sempre cria ROLE_USER; administradores são definidos pelo sistema

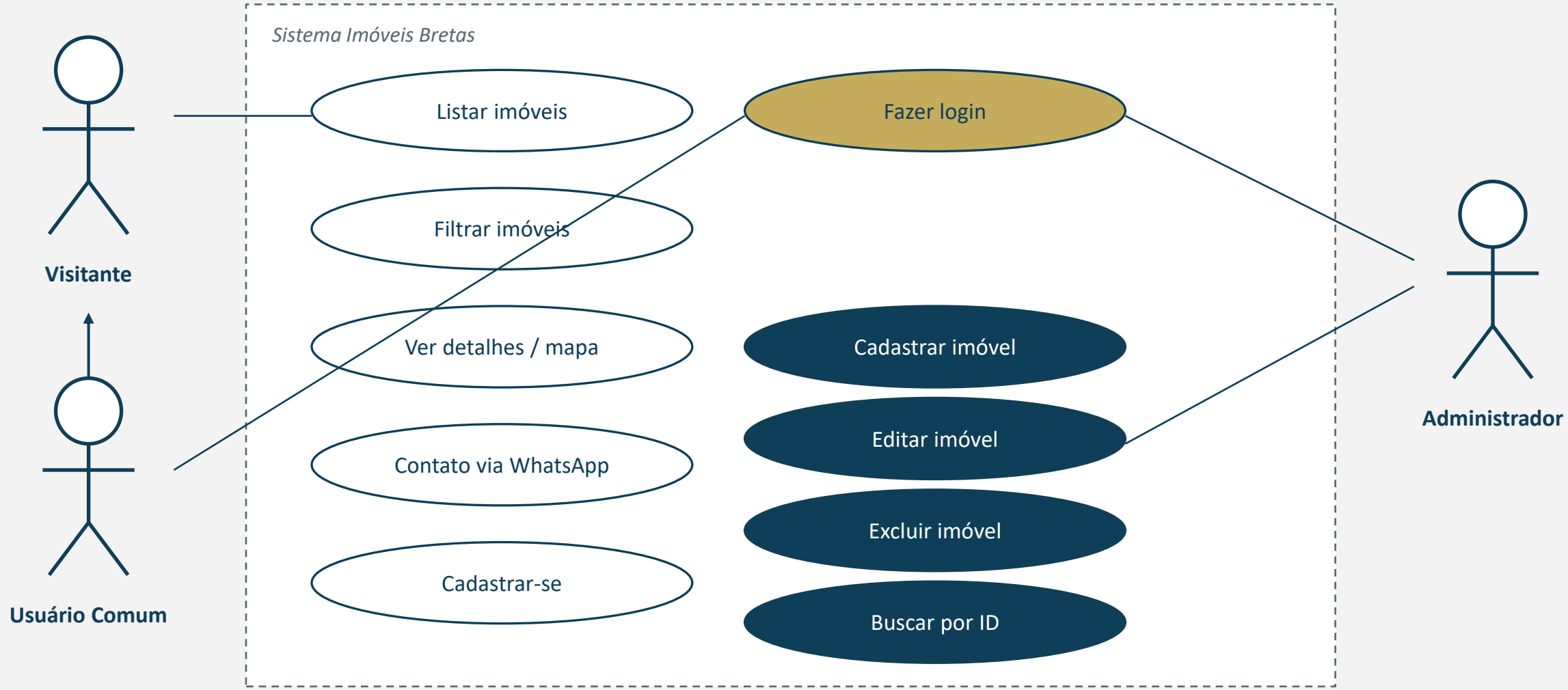
Visitante e ROLE_USER

Apenas visualizar, filtrar e ver detalhes dos imóveis.

ROLE_ADMIN

CRUD completo: cadastrar, listar, editar e excluir imóveis.

Diagrama de Casos de Uso



Usuário Comum herda os casos de uso do Visitante

Arquitetura: API REST + Front-end Separado



Back-end: Java 17 + Spring Boot (porta 8080) · Hibernate cria as tabelas a partir das entidades

Camadas e Organização de Pastas

```
backend_ib_v2/src/main/java/com/bretas/imoveis/
├── model/          Imovel, Usuario, Role
├── repository/     ImovelRepository, UsuarioRepository
├── service/        ImovelService, AuthService
├── controller/     ImovelController, AuthController
├── security/       JwtService, JwtAuthFilter
└── config/         SecurityConfig, DataSeeder

frontend_ib/src/
├── pages/          Home, AdminPage, LoginPage, PropertySelected
├── components/     CardProperty, FilterCard, Modal...
└── services/       api.js
```

1

Models / Entities

Entidades JPA mapeadas para tabelas

2

Repositories

Acesso ao banco; filtro dinâmico com Specification

3

Services

Regras de negócio e validações

4

Controllers

Endpoints REST /api/imoveis e /api/auth

Segurança com Spring Security

- SecurityFilterChain stateless com token JWT (validade de 8h)
- Senhas com BCryptPasswordEncoder
- JwtAuthFilter converte o perfil do token em ROLE_USER ou ROLE_ADMIN
- CORS liberado apenas para o front-end

Endpoint	Acesso
GET /api/imoveis, /{id}	Público
POST /api/imoveis/filtro	Público
POST /api/auth/login, /register	Público
POST /api/imoveis	ROLE_ADMIN
PUT /api/imoveis/{id}	ROLE_ADMIN
DELETE /api/imoveis/{id}	ROLE_ADMIN

401 sem login · **403** usuário sem permissão

Acessibilidade – WCAG AA

1

Contraste

Azul #0F3E58 sobre branco, acima de 4.5:1.

2

Tags semânticas

header, nav, main, section, aside e article.

3

ARIA

aria-label em botões de ícone, role="alert" em erros, aria-live na listagem.

4

Formulários e teclado

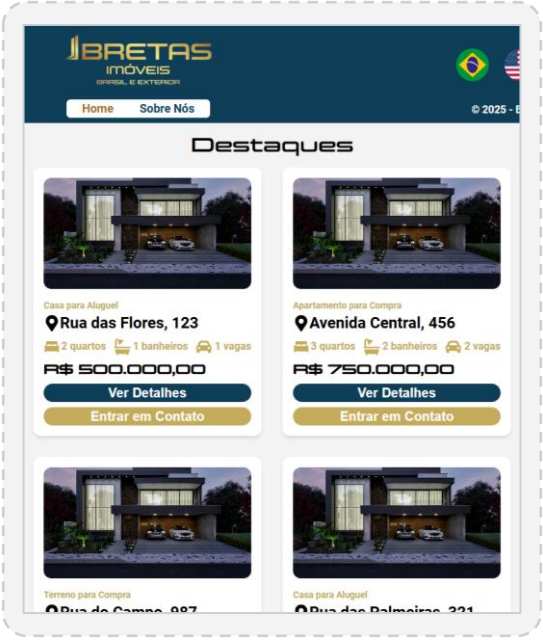
Todo campo tem label associado; ações são botões navegáveis por Tab.

5

Imagens

Textos alternativos descritivos nas fotos dos imóveis.

Interface e Fluxo de CRUD



Listagem

Home com cards e filtro lateral



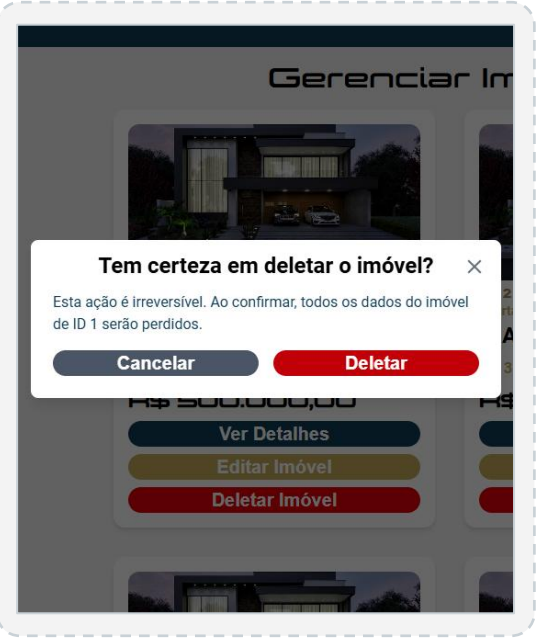
Cadastro

Modal Adicionar Imóvel (botão +)



Edição

Modal Editar com dados preenchidos



Exclusão

Modal de confirmação antes de deletar

Prints reais do sistema rodando · Também: página de detalhe (galeria, cômodos e mapa) e login do administrador

Diferenciais

Já implementados

- Filtro avançado: tipo, localização, faixa de preço e cômodos
- Contato direto por WhatsApp
- Mapa do Google Maps no detalhe
- Layout responsivo para celular

Planejados

- Upload de imagens dos imóveis
- Favoritos do usuário comum
- Paginação da listagem
- Seleção de idioma (Português / Inglês)

Divisão de Tarefas

F

Felipe

Front-end: telas React e integração com a API

J

Jeff

Banco PostgreSQL, casos de uso e documentação

R

Ruan

Back-end: entidades JPA, repositories, services e controllers

A

Anna

Design da interface, acessibilidade WCAG e slides

J

Jorge

Segurança: Spring Security, JWT e perfis de acesso

Demonstração ao vivo: projeto rodando + código na IDE